F03｜请求怎样进入，数据怎样留下

一次运行需要接通三条关系：**Windows 请求 → API listener；API → 容量服务；SQLite 文件 → 挂载存储。** 其中一条成功不替另外两条作保证。

配置写在哪，程序何时取得？

当前问题	关系或短用法	记到什么程度
同一 image 为什么能当 API，也能当容量服务？	启动入口读取 `SERVICE_ROLE`、监听地址和端口；不同实例通过环境得到不同角色。	认清 image 相同与运行配置不同可以同时成立。
`os.getenv("SERVICE_ROLE", "api")` **读到什么？**	从当前进程环境取文字；缺失时用默认值。端口文字还要转换成整数，转换失败就停止启动。	逐渐认熟“读取 → 转换 → 影响行为”；环境变量不会自己执行代码。
修改 Compose 后，旧进程为什么还用旧值？	进程已经读取启动环境；需要用新声明建立实例，restart 仍用原注入值。	**配置文件变化与进程实际配置分开。**
Compose YAML 是逐行执行的脚本吗？	它描述 services、image、environment、ports 和 volumes；缩进表达从属，`-` 表示列表项。	能看出字段属于谁；service 名称不是源码文件名。
目录里有 `.env`，程序就会拿到全部变量吗？	主机 `.env` 可供 Compose 替换 `${变量}`；进程环境还要经过镜像默认值、本次注入与覆盖规则。	分清“展开声明”和“注入进程”；不靠文件存在推断实际值。

完整声明使用配套 `compose.yaml`，不把原章的关系切片当完整运行文件。回看：「环境变量怎样变成进程行为」「Compose 把相关实例放进一份声明」。

同一个 localhost，站在哪一边看？

当前问题	关系或短用法	记到什么程度
`127.0.0.1:18080:8000` 怎样读？	主机只在 loopback 发布 18080，转到容器 8000；Windows 客户端访问前者。	逐渐认熟“主机地址 : 主机端口 : 容器端口”；两边数字不必相同。
应用监听 `0.0.0.0:8000`，客户也访问这个地址吗？	它表示在容器网络空间的各接口监听；客户使用实际可达的主机地址。	**监听范围不是客户目标地址。** 容器内仅监听 127.0.0.1 时，发布请求可能到不了该 listener。
API 为什么访问 `fake-dependency:9000`？	同一 Compose 网络按 service 名称解析当前容量实例，使用内部端口；不必把容量端口发布到 Windows。	认清内部服务交接；主机 18080 不参与这一次调用。
API 的 `localhost:9000` 指容量服务吗？	它指 API 容器自身；容量容器与 Windows 也各有自己的 localhost。	**同一写法取决于所在网络空间。**
inspect 查到 IP 后，可以写死到源码吗？	重建后 IP 可能变化，服务名可重新解析当前实例；旧连接失败仍需处理与重新连接。	知道名称稳定不等于连接永远有效。
`depends_on: service_healthy` 能保证依赖一直好吗？	配套用它等待初始健康条件；容量探针测 `/live`，以后 `/capacity` 仍可能 503。	分清启动顺序与持续业务能力，不把依赖图当永久保证。

容量调用在 `/capacity` 上做有界 GET。本例只核对 JSON 对象和 `status == "available"`，`slots` 不参与决策，因此不能说已验证全部容量字段合同。回看：「18080 与 8000 分别属于谁」「API 为什么用服务名称，而不用 localhost」。

文件名、挂载点和 volume，哪一个保存工单？

当前问题	关系或短用法	记到什么程度
`/data/tickets.db` 与 `ticket-data:/data` 怎样连起来？	应用打开前一个文件；后一个把 named volume 挂到 `/data`，该目录中的写入进入挂载存储。	沿“文件 → 挂载点 → 存储对象”追踪；三者不是同一名称。
数据写在 `/app/tickets.db`，为什么换实例后没了？	未挂载该位置时，运行中新增文件属于实例可写层；停止再启动同一实例通常保留，删除后重建不会带回。	**同实例重启和替换实例不同。**
named volume 与 bind mount 何时选？	本例数据库用 Docker 管理的 named volume；操作者要取得备份，用明确主机目录挂到 `/backups`。	按数据用途选择；Windows 路径与容器路径分别核对。
Compose 里写 `ticket-data`，实际对象就叫这个吗？	它是逻辑名，实际名称通常带 project 前缀；按 inspect 和项目 label 辨认。	不猜 Docker Desktop 的内部磁盘目录，不靠相似名称认对象。
有 volume，误删数据也能恢复吗？	它解决跨实例保留；应用误删、删除 volume 或底层损坏仍可能丢数据。	**持久存储不等于独立备份。** F05 处理恢复。

回看：「SQLite 写入的文件实际属于谁」。

声明可解析以后，还缺哪一段证据？

当前问题	关系或短用法	记到什么程度
`docker compose config --quiet` 成功，能直接说启动成功吗？	它只检查声明能解析；不证明端口空闲、进程可启动或数据库可写。	认熟静态检查的结论范围。
Compose 等健康与主机 HTTP 检查重复吗？	前者等待所配置的运行/健康条件；后者经过 Windows 发布端口再访问 `/ready`。	根据观察位置选检查；内部成功尚未覆盖主机入口。
怎样证明替换实例后保留了原数据？	对比旧/新容器 ID、实际 mount，再 GET 原 T-42 并核对字段。	**新建成功不能代替旧数据读回。** 新工单会掩盖原数据是否仍在。

启动会创建运行对象；首次 UAT 会写入，必须接上已确认的空基线；force-recreate 会替换实例。完整顺序、等待参数和失败处理回看「怎样用一次运行把声明与结果接起来」及 F06，不从表中拼一套立即执行的命令。